



UNIVERSIDADE FEDERAL DA BAHIA
ENGENHARIA DE SOFTWARE I - MATA62 - T01

FELIPE SILVA DA ANUNCIAÇÃO
JAEDSON BARBOSA MACEDO
LUCAS PEREIRA RAPOZO
NADSON DE SANTANA SOUSA

TRABALHO I: ESTILO ARQUITETURAL MICROKERNEL

SALVADOR
2026

TRABALHO I: ESTILO ARQUITETURAL MICROKERNEL

Análise dos sistemas open source Visual Studio Code e OBS Studio

Trabalho I apresentado à disciplina de Engenharia de Software I, do curso de Bacharelado em Ciência da Computação da Universidade Federal da Bahia (UFBA), como requisito de avaliação parcial.

Docente: Prof. Dr. Eduardo Almeida.

Equipe 03:

Felipe Silva da Anunciação;

Jaedson Barbosa Macedo;

Lucas Pereira Rapozo;

Nadson de Santana Sousa.

SUMÁRIO

SUMÁRIO.....	2
1. VISUAL STUDIO CODE (VS CODE).....	4
1.1 DECISÕES ARQUITETURAIS.....	4
1.2 DIAGRAMA DE MÓDULOS DA ARQUITETURA.....	5
1.3 CARACTERÍSTICAS ARQUITETURAIS E TÁTICAS.....	6
1.4 CARACTERÍSTICAS ARQUITETURAIS.....	8
1.5 CARACTERÍSTICAS ARQUITETURAIS - TOP 4.....	10
1.6 COMPONENTES CANDIDATOS.....	11
1.7 DIAGRAMA DE COMPONENTES DA ARQUITETURA.....	13
2. OBS STUDIO.....	13
2.1 CARACTERÍSTICAS ARQUITETURAIS.....	13
2.2 CARACTERÍSTICAS ARQUITETURAIS - TOP 4.....	17
2.3 LISTA DE COMPONENTES CANDIDATOS.....	20
2.4 DIAGRAMA DE COMPONENTES DA ARQUITETURA.....	23
2.5 DECISÕES ARQUITETURAIS.....	24
2.6 DIAGRAMA DE MÓDULOS DA ARQUITETURA.....	26
2.7 TÁTICAS PARA CARACTERÍSTICAS.....	27
REFERÊNCIAS.....	29

1. VISUAL STUDIO CODE (VS CODE)

Repositório do GitHub: [microsoft/vscode: Visual Studio Code](https://github.com/microsoft/vscode)

1.1 DECISÕES ARQUITETURAIS

As decisões arquiteturais formam as regras para a construção dos sistemas. Um Architecture Decision Record (ADR) captura uma única decisão arquitetural e sua justificativa, sendo essencial para documentar o ciclo de vida do projeto. A estrutura adotada contém Título, Status, Contexto, Decisão e Consequências.

Decisão 1: Adoção do Estilo Microkernel

- **Título:** ADR-001 Adoção da Arquitetura Baseada em Plugins (Microkernel).
- **Status:** Aceito.
- **Contexto:** O sistema necessita ser altamente extensível sem causar inchaço no núcleo principal (*Core*), evitando a degradação estrutural e o antipadrão *Big Ball of Mud*.
- **Decisão:** Foi definida a utilização da arquitetura Microkernel (ou *Plugin-Based Architecture*), composta por um sistema central (Core System) e múltiplos plugins acopláveis.
- **Consequências:** Esta decisão promove um forte desacoplamento e facilita a manutenção a longo prazo, definindo um vocabulário claro de componentes.

Decisão 2: Isolamento de Extensões

- **Título:** ADR-002 Isolamento de Extensões via Extension Host.
- **Status:** Aceito.
- **Contexto:** Códigos de terceiros podem conter bugs, prejudicando a confiabilidade do sistema e bloqueando a interface principal do usuário.
- **Decisão:** Nós iremos isolar o código de customização (plugins) executando-os em um processo totalmente separado, chamado *Extension Host*, interagindo com o núcleo via chamadas remotas ou eventos assíncronos.

- **Consequências:** Esta tática garante que a falha de um plugin não derrube a interface gráfica do editor, ainda que o custo de comunicação remota introduza uma latência que não é zero.

Decisão 3: Padronização de Serviços de Linguagem

- **Título:** ADR-003 Implementação do Language Server Protocol (LSP).
- **Status:** Aceito.
- **Contexto:** Suportar múltiplas linguagens exige o desenvolvimento de lógicas complexas que não devem poluir o ambiente central (Core System).
- **Decisão:** Nós iremos utilizar o particionamento de domínio para delegar a análise sintática de código a serviços distribuídos (Language Servers), comunicando-se de forma padronizada.
- **Consequências:** Aumenta exponencialmente a reutilização, visto que os provedores das linguagens podem usar o mesmo servidor em outras ferramentas, configurando um claro "trade-off" positivo para a visão do negócio.

Decisão 4: Framework de Interface Multiplataforma

- **Título:** ADR-004 Utilização do Framework Electron.
- **Status:** Aceito.
- **Contexto:** A organização precisava construir o sistema para Windows, Linux e macOS rapidamente, necessitando equilibrar restrições orçamentárias e velocidade (Business Drivers).
- **Decisão:** Nós iremos encapsular tecnologias Web (HTML/JS/CSS) usando o Electron como casca (wrapper) para a aplicação desktop.
- **Consequências:** Reduz radicalmente a necessidade de múltiplos times de desenvolvimento nativo, em troca de um maior consumo de recursos de máquina (trade-off de performance).

1.2 DIAGRAMA DE MÓDULOS DA ARQUITETURA

O diagrama a seguir ilustra a separação física e lógica dos principais módulos do VS Code, evidenciando o isolamento entre a interface de usuário, o gerenciador de extensões e os plugins de terceiros.

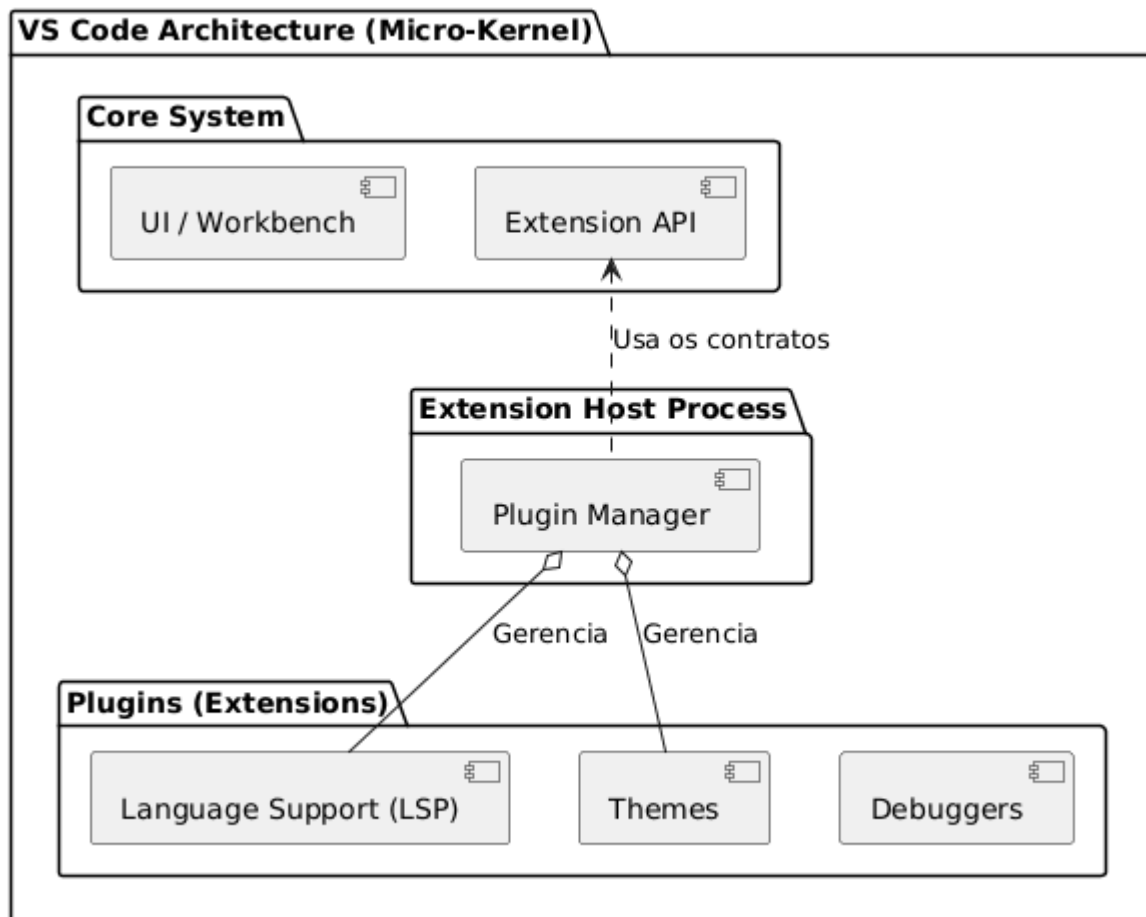


Diagrama de Módulos do VS Code

Fonte: Elaboração própria (2026).

1.3 CARACTERÍSTICAS ARQUITETURAIS E TÁTICAS

Com base na arquitetura estabelecida e nas categorias de características arquiteturais, as seguintes táticas foram aplicadas para satisfazer os requisitos do sistema:

- **Categoria Estrutural: Extensibilidade e Alavancagem/Reuso**
 - *Definição base:* Facilidade de conectar novas peças e aproveitar componentes comuns.

- *Tática Aplicada:* Utilizou-se o particionamento de componentes através do estilo Microkernel e o particionamento de domínio usando o *Language Server Protocol* (LSP).
- *Objetivo:* Permitir a adição de plugins (extensões) ilimitados sem alterar o núcleo do sistema, e reaproveitar os servidores de linguagem em outras ferramentas externas, reduzindo drasticamente o acoplamento.
- **Categoria Operacional: Tolerância a Falhas e Confiabilidade/Segurança**
 - *Definição base:* O sistema opera conforme planejado mesmo com falhas, evitando que bugs de terceiros afetem o núcleo.
 - *Tática Aplicada:* Isolamento de processos (*Process Isolation*) por meio da criação do *Extension Host*.
 - *Objetivo:* Garantir que códigos customizados de extensões sejam executados em um processo separado. Caso uma extensão gere consumo excessivo de CPU ou *crashes*, a anomalia não derrubará a interface gráfica principal (*UI/Workbench*).
- **Categoria Operacional: Desempenho (Performance)**
 - *Definição base:* Otimização dos tempos de resposta e capacidade de processamento da interface.
 - *Tática Aplicada:* Troca de mensagens assíncronas (*Asynchronous Messaging*) entre o *Core System* e os provedores de extensão.
 - *Objetivo:* Impedir que operações lentas, como compilação de código pela extensão, bloqueiem a *thread* principal do usuário, mantendo a renderização de tela e a digitação em tempo real fluida.
- **Categoria Processo: Agilidade**
 - *Definição base:* Capacidade de responder e adaptar-se rapidamente a mudanças.
 - *Tática Aplicada:* Desacoplamento da arquitetura por meio da padronização da *Extension API*.
 - *Objetivo:* Facilitar a evolução independente tanto do núcleo do editor quanto do ecossistema de extensões de terceiros, permitindo entregas

contínuas e rápidas sem necessidade de uma recompilação completa do produto.

1.4 CARACTERÍSTICAS ARQUITETURAIS

- **Extensibilidade:** É a característica que define o sistema. Quase toda funcionalidade além da edição básica de texto é entregue por extensões, inclusive recursos nativos como Git, suporte a TypeScript e Markdown. Evidência: a pasta `extensions/` do repositório contém as extensões embarcadas, que usam a mesma API pública oferecida a terceiros. O contrato dessa API está em `src/vscode-dts/vscode.d.ts`, e a documentação "Contribution Points" descreve como cada extensão declara suas contribuições no manifesto `package.json`.
- **Desempenho (Performance):** O editor precisa se manter responsivo mesmo com dezenas de extensões instaladas. Evidência: a documentação "Activation Events" mostra que as extensões só são carregadas quando necessárias, por exemplo ao abrir um arquivo de determinada linguagem. Além disso, o próprio editor oferece o comando Developer: Startup Performance, que mede o tempo de inicialização.
- **Confiabilidade e Tolerância a Falhas:** Uma falha em uma extensão não pode encerrar o editor nem causar perda de trabalho do usuário. Evidência: quando o Extension Host falha, o sistema exibe a notificação "Extension host terminated unexpectedly" e permite reiniciá-lo sem fechar a janela. O recurso Extension Bisect (blog oficial, fevereiro de 2021) localiza automaticamente a extensão defeituosa.
- **Manutenibilidade:** O código-fonte é grande e recebe contribuições de uma comunidade aberta, então precisa se manter compreensível e fácil de alterar. Evidência: a wiki "Source Code Organization" descreve uma organização em camadas (base, platform, editor, workbench), e regras de lint no repositório impedem dependências indevidas entre essas camadas.
- **Portabilidade:** O mesmo sistema executa em Windows, Linux, macOS e no navegador (vscode.dev). Evidência: o código é separado por ambiente de

execução (common, browser, node, electron-main), e a decisão pelo Electron está registrada na ADR-004.

- **Interoperabilidade e Reutilização:** O VS Code se integra a ferramentas escritas em qualquer linguagem por meio de protocolos abertos. Evidência: as especificações do Language Server Protocol e do Debug Adapter Protocol são publicadas em microsoft.github.io e adotadas por outros editores, conforme a ADR-003.
- **Segurança:** Como o sistema executa código de terceiros e abre projetos de origem desconhecida, ele precisa se proteger. Evidência: o blog "Migrating VS Code to Process Sandboxing" (novembro de 2022) descreve o sandbox da interface. O recurso Workspace Trust ativa o modo restrito (Restricted Mode) em pastas não confiáveis.
- **Configurabilidade:** O usuário ajusta praticamente todo o comportamento do editor. Evidência: as configurações são organizadas em camadas (padrão, usuário, workspace e pasta), e o editor oferece atalhos personalizáveis, temas e perfis de usuário.
- **Modularidade:** Os componentes do núcleo são desacoplados por injeção de dependências. Evidência: o serviço `IInstantiationService`, em `src/vs/platform/instantiation`, cria os serviços e resolve suas dependências a partir de identificadores declarados com `createDecorator`.
- **Agilidade (Evolutibilidade e Implantabilidade):** O sistema evolui com frequência sem quebrar o ecossistema de extensões. Evidência: há releases mensais e uma versão Insiders diária. O núcleo se atualiza automaticamente, cada extensão é versionada de forma independente, e APIs novas passam antes por uma fase de Proposed API.
- **Testabilidade:** Tanto o núcleo quanto as extensões podem ser testados de forma automatizada. Evidência: o repositório contém suítes de testes unitários, de integração e smoke tests, e a equipe publica o ferramental [@vscode/test-electron](https://github.com/microsoft/vscode-test) para autores de extensões.

- **Acessibilidade:** O editor precisa ser utilizável por pessoas com deficiência. Evidência: a documentação oficial "Accessibility" descreve o suporte a leitores de tela, os sinais sonoros (accessibility signals) e os temas de alto contraste.

1.5 CARACTERÍSTICAS ARQUITETURAIS - TOP 4

1. Extensibilidade: É a razão de ser do estilo Microkernel (ADR-001). Sem ela, o VS Code seria apenas um editor de texto. A evidência mais forte da prioridade é que a própria equipe da Microsoft implementa funcionalidades centrais como extensões, usando exatamente o mesmo contrato disponível para terceiros. Essa característica fundamenta o conjunto formado pelo registro de contribuições, pelos contribution points e pela API pública.

2. Confiabilidade e Tolerância a Falhas: O sistema executa código de milhares de autores diferentes, sem garantia de qualidade. Isso tornou obrigatório proteger o núcleo contra os próprios plugins. A característica justifica a decisão mais cara da arquitetura, o isolamento em um processo separado (ADR-002). O custo é aceito: toda chamada da API precisa ser serializada entre processos.

3. Desempenho (Performance): Um editor de código se diferencia pela sensação de leveza. Uma extensão lenta não pode travar a digitação. Essa característica motivou a ativação sob demanda das extensões e a comunicação assíncrona entre o Core System e o Extension Host. O Extension Host, portanto, atende ao mesmo tempo a esta característica e à anterior.

4. Interoperabilidade e Reutilização: Um projeto de código aberto com releases mensais corre o risco de degenerar em um Big Ball of Mud. A manutenibilidade justificou a organização em camadas com regras de dependência verificadas automaticamente. Justificou também a separação da lógica de linguagens em servidores externos via LSP (ADR-003), que ainda permite reutilizar esses servidores em outras ferramentas.

A Portabilidade e a Segurança ficaram fora do top 4, apesar de relevantes. A Portabilidade foi resolvida principalmente por uma decisão de tecnologia, o Electron (ADR-004), e não pela estrutura de componentes. A Segurança é tratada sobretudo por

táticas aplicadas sobre a estrutura já definida, como o sandbox e o Workspace Trust, e não como motivadora da estrutura em si.

1.6 COMPONENTES CANDIDATOS

Sistema Central (Core System)

- Processo Principal (src/vs/code/electron-main): Controla o ciclo de vida da aplicação: criação de janelas, integração com o sistema operacional e atualização automática. Atende à portabilidade e à agilidade.
- Workbench (src/vs/workbench): Monta a interface do usuário: layout, barra de atividades, painéis e paleta de comandos. Também oferece as áreas onde as extensões inserem suas views e menus. Atende à extensibilidade e à acessibilidade.
- Editor Monaco (src/vs/editor): Cuida da edição de texto: modelo de texto, renderização, cursores, desfazer/refazer e os provedores de linguagem, como autocompletar e hover. Atende ao desempenho.
- Serviços de Plataforma e Injeção de Dependências (src/vs/platform): Oferece serviços compartilhados, como logs, armazenamento e telemetria, e cria as instâncias dos componentes por meio do `IInstantiationService`. Atende à modularidade e à testabilidade.
- Serviço de Arquivos (IFileService): Dá acesso uniforme a arquivos por meio de provedores intercambiáveis: disco local, máquina remota ou sistema de arquivos virtual. Atende à portabilidade e à extensibilidade.
- Configuração e Perfis: Resolve as configurações em camadas e gerencia os perfis de usuário. Atende à configurabilidade.
- Comandos e Atalhos (CommandsRegistry): Mantém o registro universal de comandos, que serve de ponte entre interface, atalhos e extensões. Atende à extensibilidade e à configurabilidade.
- Workspace Trust: Avalia se um projeto é confiável e restringe funcionalidades quando não é. Atende à segurança.

- Terminal (Pty Host): Executa os terminais integrados em um processo separado. Atende à tolerância a falhas e ao desempenho.

Infraestrutura de Plugins

- Registro de Contribuições (src/vs/platform/registry): Lê os manifestos das extensões, valida e registra o que cada uma oferece. Corresponde ao registro de plugins descrito no livro. Atende à extensibilidade.
- Extension Service (src/vs/workbench/services/extensions): Descobre as extensões, decide onde cada uma será executada (local, web ou remoto), dispara os eventos de ativação e monitora o estado dos processos. Atende ao desempenho, à tolerância a falhas e à portabilidade.
- Extension Host: Processo isolado onde o código das extensões é executado (ADR-002). Atende à tolerância a falhas, ao desempenho e à segurança.
- API de Extensões e Protocolo RPC (vscode.d.ts, extHost.protocol.ts): Define o contrato público entre o núcleo e os plugins e faz a comunicação entre os processos. Atende à extensibilidade e à agilidade.
- Gerenciamento de Extensões (src/vs/platform/extensionManagement): Instala, atualiza, desinstala, habilita e desabilita extensões, consultando o Marketplace. Atende à agilidade e à segurança.
- Servidor Remoto (src/vs/server): Executa o Extension Host e os serviços de arquivos e terminal em outra máquina, no WSL ou em containers. Atende à portabilidade e à interoperabilidade.

Plugins

- Extensões Embarcadas (extensions/): Entregam funcionalidades padrão, como Git, TypeScript, Markdown e temas. Atendem à extensibilidade.
- Extensões de Terceiros: Adicionam novas capacidades distribuídas pelo Marketplace. Atendem à extensibilidade.
- Servidores de Linguagem (LSP): Processos externos que fornecem análise de código para cada linguagem (ADR-003). Atendem à interoperabilidade e à reutilização.

- Adaptadores de Depuração (DAP): Integram depuradores específicos de cada linguagem por meio de um **protocolo padronizado**. **Atendem à interoperabilidade**.

1.7 DIAGRAMA DE COMPONENTES DA ARQUITETURA

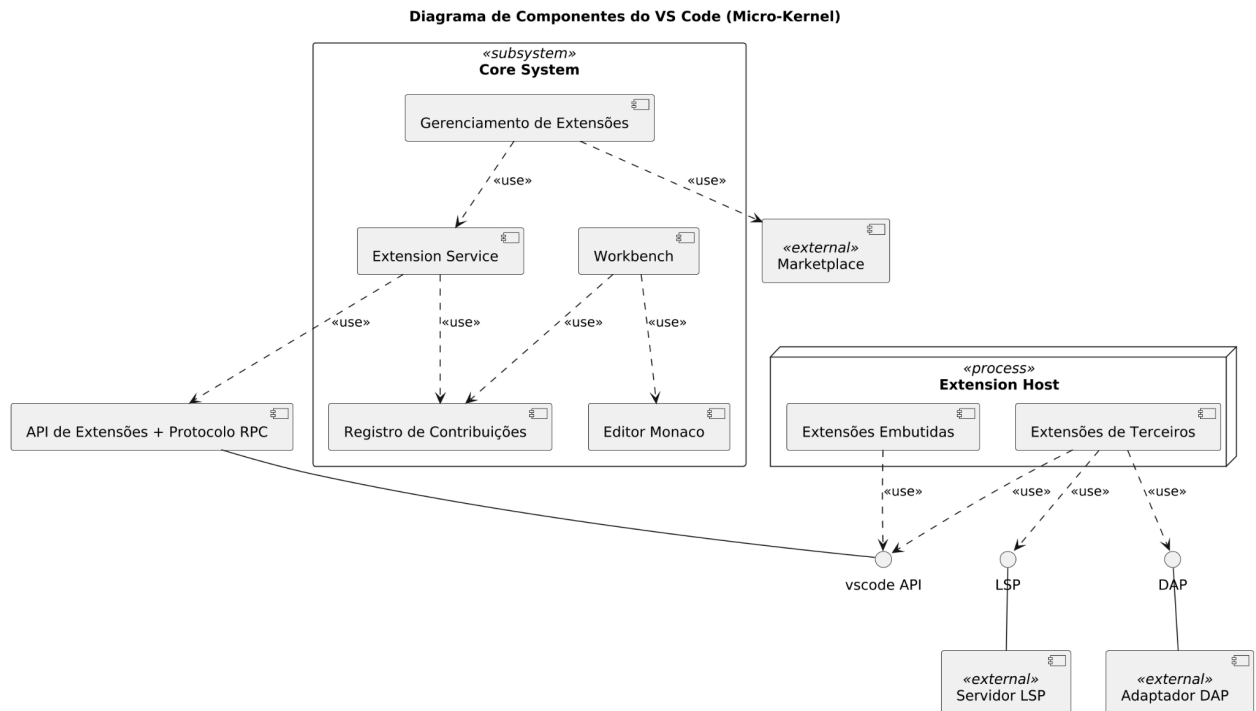


Diagrama de Componentes do VS Code

Fonte: Elaboração própria (2026).

2. OBS STUDIO

Repositório do GitHub: [obsproject/obs-studio: OBS Studio - Free and open source software for live streaming and screen recording](https://github.com/obsproject/obs-studio)

2.1 CARACTERÍSTICAS ARQUITETURAIS

Característica: Agilidade

- **Título:** ADR-001 Adoção do Estilo Microkernel para Favorecer a Agilidade.
- **Status:** Aceito.
- **Contexto:** O sistema exige suporte contínuo a novos hardwares e plataformas de *streaming*, precisando responder rapidamente a mudanças e ser mantido por centenas de colaboradores *open source* sem degradar o núcleo principal.
- **Decisão:** Foi definida a priorização da agilidade por meio da adoção do estilo Microkernel, que estabelece o isolamento de responsabilidades e a criação de contratos de API claros.
- **Consequências:** A comunidade de desenvolvedores e os fabricantes podem criar módulos de forma independente. O código se mantém legível e modular, e novos *plugins* não quebram o motor central.
- **Evidências:** O diretório *plugins/* do repositório abriga as funcionalidades periféricas, e a estrutura de diretórios separa a interface (diretório *frontend/*) do núcleo (*libobs*), que oferece uma API em C (*obs-module.h*).

Característica: Desempenho

- **Título:** ADR-002 Priorização do Desempenho no Processamento Multimídia.
- **Status:** Aceito.
- **Contexto:** Atrasos ou bloqueios nos processos de vídeo resultam em perda de quadros durante o processamento em tempo real.
- **Decisão:** Garantir comunicação de baixa latência entre o sistema central e os *plugins* de codificação, minimizando a sobrecarga causada pela cópia de memória.
- **Consequências:** Transmissão ao vivo fluida. O sistema consegue sustentar altas taxas de atualização em tempo real sem esgotar o processador do computador.
- **Evidências:** O núcleo (*libobs*) foi desenvolvido predominantemente em linguagem C para maximizar a velocidade. Observa-se também o compartilhamento de texturas diretamente na memória da GPU.

Característica: Confiabilidade e Tolerância a Falhas

- **Título:** ADR-003 Priorização da Confiabilidade e da Tolerância a Falhas em Ambientes de *Broadcasting*.

- **Status:** Aceito.
- **Contexto:** Durante uma transmissão ao vivo, interrupções ou falhas no *pipeline* central de áudio e vídeo causam grande impacto para o usuário. Além disso, erros de hardware ou falhas de rede não podem causar o encerramento ou o bloqueio de toda a aplicação durante uma gravação.
- **Decisão:** Isolar estritamente o *loop* principal de renderização multimídia, de modo que o núcleo não dependa do tempo de resposta de serviços externos, e executar de forma assíncrona os módulos propensos a falhas externas, com tratamento local de exceções.
- **Consequências:** Garante a continuidade do serviço básico, sem congelar nem corromper a captura principal, mesmo sob uso intenso. Se a rede cair, o *plugin* de saída tenta se reconectar de forma autônoma, sem bloquear a renderização nem congelar a interface gráfica.
- **Evidências:** A implementação do *libobs*, que gerencia a orquestração e a sincronização de *threads* de forma independente dos *plugins* periféricos, e as *threads* assíncronas distintas para a comunicação de rede e a captura de dispositivos, separadas do *loop* principal.

Característica: Capacidade de Aprendizado

- **Título:** ADR-004 Foco na Capacidade de Aprendizado por meio de uma Interface Gráfica Rica.
- **Status:** Aceito.
- **Contexto:** O processamento de áudio e vídeo em tempo real é complexo, mas o usuário final precisa de ferramentas visuais, rápidas e intuitivas para aprender a operar o sistema sem dificuldades.
- **Decisão:** Construir um *frontend* interativo e dedicado, capaz de fornecer *feedback* visual em tempo real e mantido como uma camada totalmente separada do motor de processamento.
- **Consequências:** Reduz a curva de aprendizado para usuários iniciantes. Permite operações visuais complexas por meio de ações simples, evitando que a operação do sistema sobrecarregue o usuário final.

- **Evidências:** A implementação dos componentes na *framework* Qt, que oferece painéis modulares, o "Modo Estúdio" para transições seguras e a manipulação de fontes por arrastar e soltar.

Característica: Implantabilidade e Portabilidade

- **Título:** ADR-005 Maximização da Implantabilidade Multiplataforma.
- **Status:** Aceito.
- **Contexto:** O software precisa atender usuários de diferentes sistemas operacionais e ser distribuído de forma consistente em Windows, macOS e Linux.
- **Decisão:** Isolar e abstrair as chamadas nativas de cada sistema operacional (gerenciamento de janelas, E/S e renderização gráfica) e utilizar o carregamento dinâmico de bibliotecas.
- **Consequências:** Facilita a implantação, permitindo que o sistema seja empacotado em instaladores independentes e que novas funcionalidades específicas de cada sistema operacional sejam adicionadas sem poluir o núcleo.
- **Evidências:** A separação física dos pacotes no código-fonte (*win-capture*, *mac-capture* e *linux-capture*) e a adoção da *framework* Qt para uma UI unificada.

Característica: Extensibilidade

- **Título:** ADR-006 Extensibilidade por meio do Registro Padronizado de Módulos.
- **Status:** Aceito.
- **Contexto:** O ecossistema de captura, codificação e transmissão muda continuamente, com novos dispositivos, codecs e plataformas de *streaming*. A equipe central não consegue suportar tudo, e a comunidade e os fabricantes precisam adicionar funcionalidades sem alterar o núcleo.
- **Decisão:** Expor no *libobs* uma API de registro de módulos, na qual cada *plugin* declara, ao ser carregado, as fontes, os filtros, os codificadores e as saídas que oferece. O núcleo apenas consome essas capacidades por meio de contratos padronizados.
- **Consequências:** Novos tipos de fonte, codificador e saída podem ser adicionados sem modificar o núcleo, e a maior parte das funcionalidades visíveis

ao usuário é entregue como módulos. Em contrapartida, o núcleo precisa manter a compatibilidade da API, e os módulos são executados no mesmo processo dele.

- **Evidências:** O diretório *plugins/* reúne os módulos que acompanham o OBS (como *obs-x264*, *obs-ffmpeg* e *obs-outputs*) e usa o mesmo contrato oferecido a terceiros: o cabeçalho *obs-module.h*, a função *obs_module_load()* e as funções de registro *obs_register_source()*, *obs_register_encoder()* e *obs_register_output()*.

Característica: Testabilidade

- **Título:** ADR-007 Separação de Módulos para Alta Testabilidade.
- **Status:** Aceito.
- **Contexto:** No contexto do OBS, é essencial garantir que novas funcionalidades e refatorações possam ser validadas automaticamente, para não quebrarem a lógica existente antes de cada lançamento.
- **Decisão:** Manter forte desacoplamento entre o núcleo (*libobs*), a interface gráfica e os *plugins* externos.
- **Consequências:** Promove alta testabilidade, permitindo que a lógica de negócio central e o processamento de quadros sejam testados de forma independente do estado da interface gráfica ou do hardware.
- **Evidências:** O isolamento garantido pelos cabeçalhos da API em C, que permite invocar e simular funcionalidades do núcleo sem inicializar o *frontend* Qt.

2.2 CARACTERÍSTICAS ARQUITETURAIS - TOP 4

Característica: Extensibilidade

- **Título:** ADR-006 Extensibilidade por meio do Registro Padronizado de Módulos.
- **Status:** Aceito.
- **Contexto:** A comunidade e os fabricantes precisam adicionar novos dispositivos, codecs e plataformas sem alterar o núcleo.

- **Decisão:** Expor no *libobs* uma API de registro de módulos, na qual cada *plugin* declara, ao ser carregado, as fontes, os filtros, os codificadores e as saídas que oferece.
- **Consequências:** Novos tipos de fonte, codificador e saída podem ser adicionados sem modificar o núcleo. Em contrapartida, o núcleo precisa manter a compatibilidade da API.
- **Evidências:** O diretório *plugins/* e o contrato *obs-module.h*, com a função *obs_module_load()* e as funções de registro de fontes, codificadores e saídas.

Característica: Desempenho

- **Título:** ADR-002 Priorização do Desempenho no Processamento Multimídia.
- **Status:** Aceito.
- **Contexto:** Atrasos ou bloqueios nos processos de vídeo resultam em perda de quadros durante o processamento em tempo real.
- **Decisão:** Garantir comunicação de baixa latência entre o sistema central e os *plugins* de codificação, minimizando a sobrecarga causada pela cópia de memória.
- **Consequências:** Transmissão ao vivo fluida. O sistema consegue sustentar altas taxas de atualização em tempo real.
- **Evidências:** O núcleo (*libobs*) foi desenvolvido predominantemente em linguagem C para maximizar a velocidade.

Característica: Confiabilidade e Tolerância a Falhas

- **Título:** ADR-003 Priorização da Confiabilidade e da Tolerância a Falhas em Ambientes de *Broadcasting*.
- **Status:** Aceito.
- **Contexto:** Durante uma transmissão ao vivo, interrupções ou falhas no *pipeline* central de áudio e vídeo causam grande impacto para o usuário. Além disso, erros de hardware ou falhas de rede não podem encerrar nem bloquear a aplicação durante uma gravação.

- **Decisão:** Isolar estritamente o *loop* principal de renderização, de modo que o núcleo não dependa do tempo de resposta de serviços externos, e executar de forma assíncrona os módulos propensos a falhas externas, com tratamento local de exceções.
- **Consequências:** Continuidade do serviço básico, sem congelar nem corromper a captura, mesmo sob uso intenso. Em caso de queda da rede, o *plugin* de saída tenta se reconectar de forma autônoma.
- **Evidências:** A implementação do *libobs*, que gerencia a orquestração e a sincronização de *threads* de forma independente dos *plugins* periféricos, e as *threads* assíncronas para rede e captura.

Característica: Implantabilidade e Portabilidade

- **Título:** ADR-005 Maximização da Implantabilidade Multiplataforma.
- **Status:** Aceito.
- **Contexto:** O software precisa atender usuários de diferentes sistemas operacionais e ser distribuído de forma consistente em Windows, macOS e Linux.
- **Decisão:** Isolar e abstrair as chamadas nativas de cada sistema operacional e utilizar o carregamento dinâmico de bibliotecas.
- **Consequências:** Facilita a implantação, permitindo que o sistema seja empacotado em instaladores independentes.
- **Evidências:** A separação física dos pacotes no código-fonte (*win-capture*, *mac-capture* e *linux-capture*) e a adoção da *framework* Qt para uma UI unificada.

Justificativa da escolha do top 4

1. Extensibilidade: É a razão de ser do estilo Microkernel. Sem ela, o OBS seria apenas um compositor de cenas com um conjunto fixo de fontes e codificadores. A evidência mais forte é que as próprias funcionalidades padrão, como captura, codificação e saída, são entregues como módulos em *plugins/*, pelo mesmo contrato (*obs-module.h*) oferecido a terceiros. Essa característica fundamenta a Plugin Interface e os Plugins de Fontes, de Codificação e de Saída.

2. Desempenho: Transmitir e gravar em tempo real a 30 ou 60 quadros por segundo não admite atrasos, e um quadro perdido é percebido de imediato. Essa característica motivou a implementação do núcleo em C, o compartilhamento de texturas na GPU sem cópia (ADR-IMP-003) e a comunicação de baixa latência com os *plugins* de codificação. É também a que mais restringe os *plugins*, que precisam executar no contexto gráfico controlado pelo núcleo.

3. Confiabilidade e Tolerância a Falhas: Uma transmissão ao vivo não pode ser repetida: se o programa travar ou a gravação for corrompida, o conteúdo se perde. Como o sistema depende de redes instáveis e de dispositivos de terceiros, a falha externa é esperada, e não excepcional. Isso justificou isolar o *loop* de renderização e executar rede e captura em *threads* próprias, com reconexão automática. Como os módulos são carregados no mesmo processo do núcleo, a proteção vem do tratamento local de erros e da separação de *threads*, e não do isolamento de processos, como no VS Code.

4. Implantabilidade e Portabilidade: O OBS é distribuído para Windows, macOS e Linux e precisa receber novos módulos sem recompilar o núcleo. Essa característica justificou a abstração das chamadas nativas do sistema operacional, a separação dos módulos de captura por plataforma (*win-capture*, *mac-capture* e *linux-capture*) e o carregamento dinâmico de bibliotecas (ADR-IMP-004).

A Agilidade, a Testabilidade e a Capacidade de Aprendizado ficaram fora do top 4, apesar de relevantes. A Agilidade é, em grande parte, consequência da Extensibilidade: como os módulos evoluem de forma independente, a comunidade entrega mudanças sem alterar o núcleo. A Testabilidade decorre do desacoplamento que o próprio estilo já exige, sendo um benefício da estrutura mais do que sua motivação. A Capacidade de Aprendizado é resolvida na camada de interface (Qt), que pode ser substituída sem afetar o motor, e por isso não determina a estrutura de componentes do sistema.

2.3 LISTA DE COMPONENTES CANDIDATOS

- **Microkernel (*libobs*)**

- **Responsabilidades:** Orquestrar o *pipeline* de áudio e vídeo, gerenciar o ciclo de vida da aplicação e carregar e descarregar os *plugins* em tempo de execução.
- **Requisitos do Sistema:** Satisfaz a necessidade de um gerenciador centralizado que sincronize com precisão as faixas de áudio e vídeo capturadas antes de enviá-las para codificação.
- **Características Arquiteturais:** Garante o Desempenho, por ser otimizado em C para latência mínima, e atua como pilar da Confiabilidade, protegendo a orquestração central de falhas externas.
- **Plugin Interface (*obs-module*)**
 - **Responsabilidades:** Definir as assinaturas de funções e os cabeçalhos que determinam como um *plugin* se registra, informa suas capacidades e se comunica com o *libobs*.
 - **Requisitos do Sistema:** Cumpre o requisito de padronizar e validar a comunicação segura entre código de terceiros e o motor principal.
 - **Características Arquiteturais:** Representa a Extensibilidade, permitindo o acoplamento de novos módulos sem alterar o código-fonte do núcleo.
- **Interface do Usuário (UI - Qt Frontend)**
 - **Responsabilidades:** Fornecer a interface gráfica (GUI) que permite ao usuário gerenciar cenas e fontes e iniciar transmissões, funcionando como um cliente que envia comandos ao *libobs*.
 - **Requisitos do Sistema:** Atende à necessidade de fornecer aos usuários finais um painel de controle visual, intuitivo e reativo.
 - **Características Arquiteturais:** Representa a Portabilidade, pois o uso da *framework* Qt permite compilar a mesma interface nativamente para Windows, macOS e Linux.
- **Plugins de Fontes (Source Plugins)**

- **Responsabilidades:** Capturar dados brutos de dispositivos locais ou do sistema operacional.
- **Requisitos do Sistema:** Resolve a necessidade de suportar uma ampla gama de dispositivos e métodos de entrada de imagem e som.
- **Características Arquiteturais:** Impulsiona a Extensibilidade e respeita a Portabilidade por meio da separação em submódulos, como *win-capture* ou *linux-capture*.

- **Plugins de Codificação (Encoder Plugins)**

- **Responsabilidades:** Receber os *frames* compostos pelo núcleo e comprimi-los em formatos de vídeo e áudio padronizados.
- **Requisitos do Sistema:** Atende à necessidade de reduzir o tamanho dos arquivos de vídeo brutos para que possam ser transmitidos com eficiência pela internet.
- **Características Arquiteturais:** Direcionado ao Desempenho, integra-se às APIs da GPU para realizar a compressão sem sobrecarregar o processador central.

- **Plugins de Saída (Output Plugins)**

- **Responsabilidades:** Empacotar e enviar os dados codificados ao destino final, seja gravando em um arquivo local ou transmitindo via protocolo RTMP.
- **Requisitos do Sistema:** Cumpre o requisito principal do software: publicar a transmissão em plataformas de *streaming* externas ou arquivá-la localmente.
- **Características Arquiteturais:** Essencial para a Confiabilidade, pois opera em *threads* assíncronas dedicadas à rede. Em caso de perda de pacotes, gerencia as tentativas de reconexão sem congelar o restante do software.

2.4 DIAGRAMA DE COMPONENTES DA ARQUITETURA

O diagrama ilustra os componentes mais críticos do estilo Microkernel. O *libobs* e a API *obs-module* foram selecionados por serem o eixo central da arquitetura. Dentre o vasto ecossistema de extensões, foram documentados os *Source Plugins*, *Encoder Plugins* e *Output Plugins*, pois representam coletivamente o *pipeline* mínimo e essencial de funcionamento do software: capturar a imagem, comprimir os dados e enviá-los para a rede.

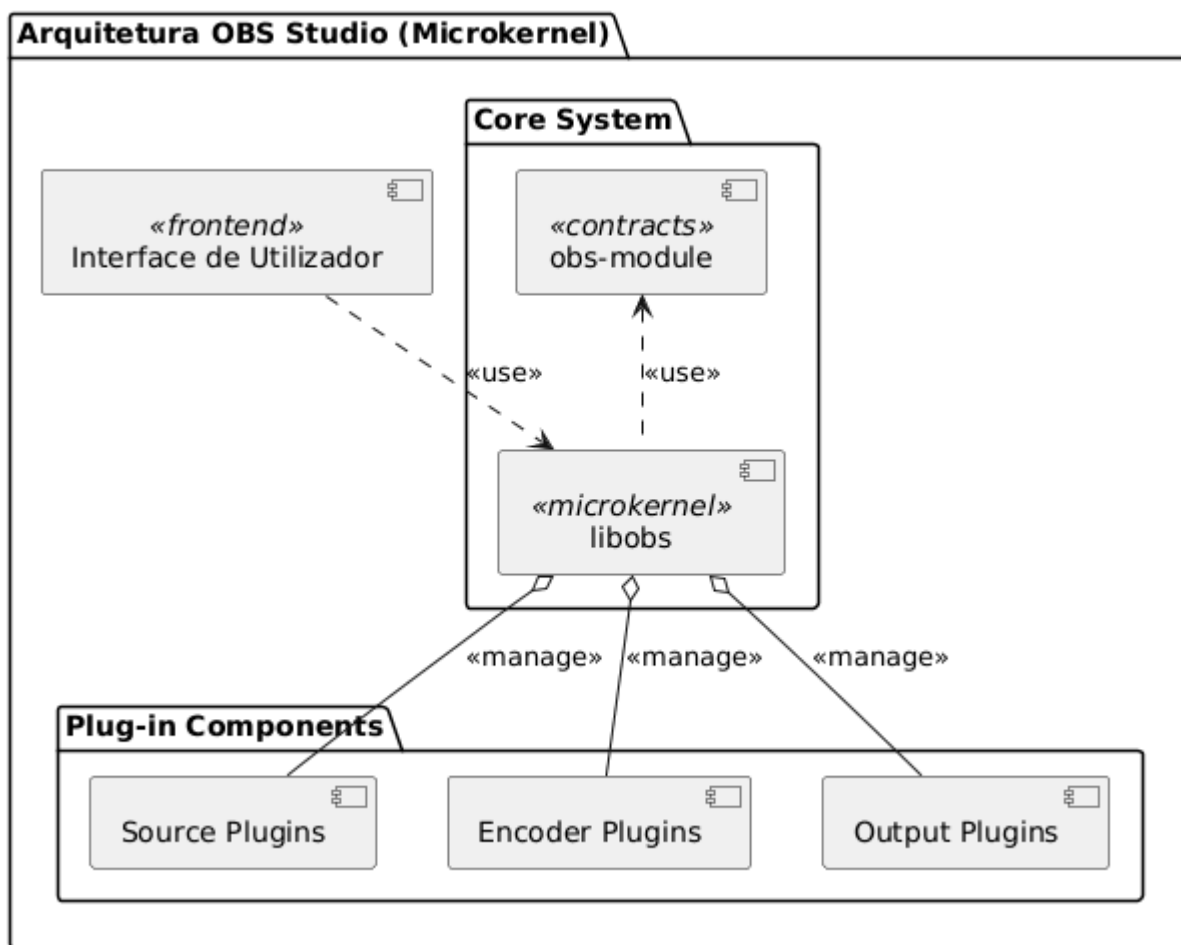


Diagrama de Componentes do OBS Studio

Fonte: Elaboração própria (2026).

2.5 DECISÕES ARQUITETURAIS

Decisão Arquitetural 1: Implementação do Núcleo (Microkernel) em Linguagem C

- **Título:** ADR-IMP-001 Adoção da Linguagem C para o Núcleo (*libobs*).
- **Status:** Aceito.
- **Contexto:** O motor de coordenação de áudio e vídeo precisa de controle sobre a gestão de memória, as *threads* e os ciclos de relógio do processador, além de exportar uma Interface Binária de Aplicação (ABI) estável, para que *plugins* desenvolvidos em outras linguagens possam se ligar a ele.
- **Decisão:** Foi decidido desenvolver o núcleo *libobs* em linguagem C, em vez de C++ ou outra linguagem de mais alto nível.
- **Consequências:** Favorece alto desempenho e baixa latência. A ABI do C facilita o desenvolvimento de *plugins* e de interfaces de usuário, mantendo o sistema central agnóstico em relação à tecnologia periférica.
- **Evidências:** Ao analisar o repositório, vê-se que o diretório *libobs/* é composto predominantemente por arquivos *.c* e cabeçalhos *.h*, evitando classes e bibliotecas padrão do C++ no núcleo do sistema.

Decisão Arquitetural 2: Separação Física da Interface do Usuário

- **Título:** ADR-IMP-002 Desacoplamento da Interface Visual por meio da *Framework Qt*.
- **Status:** Aceito.
- **Contexto:** Em um software multiplataforma, o código de janelas e controles visuais tende a ficar intrinsecamente ligado à lógica de negócio se não houver cuidado. Além disso, empresas terceiras podem querer usar o motor do OBS com interfaces próprias.
- **Decisão:** Foi decidido isolar completamente o desenvolvimento da interface do usuário, adotando C++ e a *framework Qt* para a UI, que se comunica com o núcleo apenas por meio de chamadas à API exportada (*obs.h*).

- **Consequências:** Permite a construção de clientes alternativos. A UI original em Qt pode ser atualizada ou substituída sem alterar uma única linha do código de processamento de vídeo.
- **Evidências:** O código-fonte apresenta um diretório *frontend/* distinto, escrito em C++ com arquivos *.ui* do Qt, que faz chamadas à API do *libobs*, sem lógica interna de orquestração multimídia.

Decisão Arquitetural 3: Passagem de Texturas em GPU (Zero-Copy) entre Módulos

- **Título:** ADR-IMP-003 Compartilhamento de Contexto Gráfico para Processamento de Vídeo.
- **Status:** Aceito.
- **Contexto:** Copiar os *frames* de vídeo da memória RAM para a placa gráfica e vice-versa, a cada filtro ou transformação aplicada, reduziria o desempenho do sistema e dificultaria a transmissão a 60 FPS.
- **Decisão:** Foi decidido implementar um sistema de contexto compartilhado de renderização. Os *plugins* passam entre si apenas as referências das texturas armazenadas na memória da GPU, evitando a cópia dos pixels.
- **Consequências:** Redução do uso de CPU e de RAM do sistema. Em contrapartida, limita os desenvolvedores de *plugins* visuais, que precisam executar seu código no contexto das *threads* gráficas controladas pelo OBS.
- **Evidências:** A existência e o uso intensivo da estrutura *gs_texture_t* e das funções da API gráfica (*Graphics Subsystem*) nos cabeçalhos C do *libobs*, que obrigam os módulos a se sincronizar com a GPU.

Decisão Arquitetural 4: Carregamento Dinâmico de Módulos

- **Título:** ADR-IMP-004 Injeção de Funcionalidades via Carregamento Dinâmico do Sistema Operacional.
- **Status:** Aceito.
- **Contexto:** A arquitetura Microkernel exige que o sistema central carregue os *plugins* sem precisar ser recompilado sempre que um usuário instala um novo módulo de terceiros.

- **Decisão:** Foi decidido utilizar as APIs nativas de carregamento dinâmico de bibliotecas de cada sistema operacional para ler arquivos binários presentes em uma pasta específica durante a inicialização.
- **Consequências:** Permite uma arquitetura verdadeiramente *plug and play*. O núcleo inicia, varre as pastas de *plugins* (como *obs-plugins/*), carrega as bibliotecas em tempo de execução e registra os *callbacks* (funções de retorno) de cada *plugin*.
- **Evidências:** A implementação da lógica de carregamento multiplataforma no arquivo *libobs/obs-module.c*, que define como os módulos externos são carregados na memória e iniciados pela função padronizada *obs_module_load()*, implementada por cada módulo.

2.6 DIAGRAMA DE MÓDULOS DA ARQUITETURA

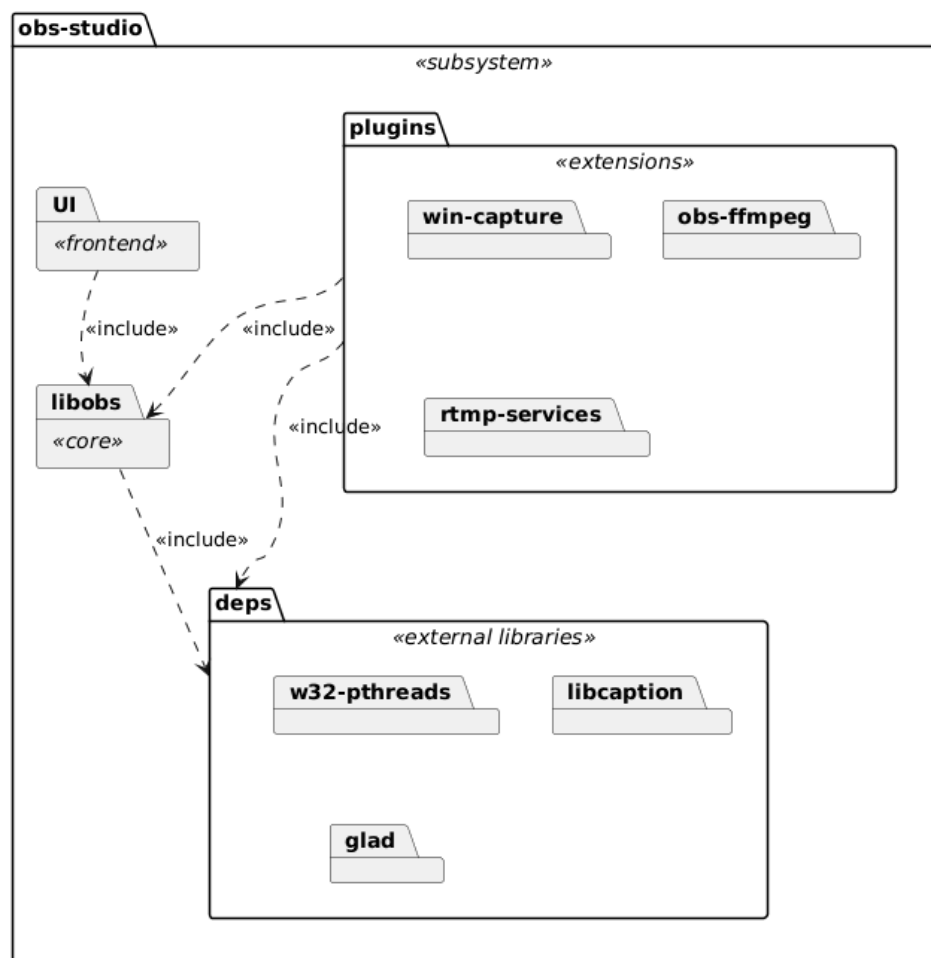


Diagrama de Módulos do OBS Studio

Fonte: Elaboração própria (2026).

2.7 TÁTICAS PARA CARACTERÍSTICAS

- **Implantabilidade:** Utiliza o *binding* em tempo de execução para permitir a descoberta e o carregamento das bibliotecas de *plugins* apenas no momento da execução, facilitando a criação de instaladores independentes e a atualização modular do sistema sem a necessidade de recompilar o núcleo.
- **Desempenho (Performance):** Aplica a tática de aumento da eficiência de recursos, por meio de operações *zero-copy* e aceleração de *hardware*, para reduzir a latência e a sobrecarga do processador, compartilhando referências de texturas de vídeo diretamente na memória da placa gráfica e evitando cópias custosas de pixels brutos.
- **Confiabilidade e Tolerância a Falhas:** Recorre ao isolamento de falhas, à execução assíncrona e ao tratamento local de exceções, com o objetivo de garantir que operações propensas a atrasos ou interrupções externas ocorram em *threads* separadas e que erros imprevistos em um módulo sejam tratados no próprio módulo, impedindo que a falha congele o *loop* principal de orquestração e a interface gráfica ou encerre abruptamente a aplicação.
- **Testabilidade:** Emprega o desacoplamento e a abstração de interfaces para isolar estritamente a lógica de negócio das dependências da interface gráfica (Qt) e dos dispositivos físicos, o que permite criar ambientes de teste nos quais componentes e fontes de áudio e vídeo possam ser facilmente simulados e validados de forma autônoma.
- **Agilidade:** Aplica o encapsulamento e interfaces padronizadas para estabelecer fronteiras rígidas de comunicação, permitindo que a comunidade adicione, modifique e integre novos *plugins* rapidamente, sem risco de corromper o núcleo central.
- **Capacidade de Aprendizado:** Recorre à separação da interface do usuário para manter a lógica de interação visual estritamente na *framework* Qt, com o objetivo

de criar painéis de controle intuitivos, o "Modo Estúdio" e *feedbacks* visuais que reduzam a curva de aprendizado para criadores de conteúdo sem conhecimento técnico.

- **Extensibilidade:** Aplica o registro de módulos por meio de uma API padronizada (*obs-module.h*), na qual cada *plugin* declara, na carga (*obs_module_load()*), as fontes, os codificadores e as saídas que oferece, com o objetivo de permitir a inclusão de novas capacidades sem alterar nem recompilar o núcleo, que permanece responsável apenas pela orquestração.

REFERÊNCIAS

MICROSOFT. **Accessibility**. In: Visual Studio Code Documentation. Disponível em: <https://code.visualstudio.com/docs/editor/accessibility>. Acesso em: 29 set. 2026.

MICROSOFT. **Activation Events**. In: Visual Studio Code Extension API. Disponível em: <https://code.visualstudio.com/api/references/activation-events>. Acesso em: 29 set. 2026.

MICROSOFT. **Contribution Points**. In: Visual Studio Code Extension API. Disponível em: <https://code.visualstudio.com/api/references/contribution-points>. Acesso em: 29 set. 2026.

MICROSOFT. **Debug Adapter Protocol**. Especificação. Disponível em: <https://microsoft.github.io/debug-adapter-protocol/>. Acesso em: 29 set. 2026.

MICROSOFT. **Extension Host**. In: Visual Studio Code Extension API. Disponível em: <https://code.visualstudio.com/api/advanced-topics/extension-host>. Acesso em: 29 set. 2026.

MICROSOFT. **Language Server Protocol**. Especificação. Disponível em: <https://microsoft.github.io/language-server-protocol/>. Acesso em: 29 set. 2026.

MICROSOFT. **Source Code Organization**. Wiki do repositório microsoft/vscode. Disponível em: <https://github.com/microsoft/vscode/wiki/Source-Code-Organization>. Acesso em: 29 set. 2026.

MICROSOFT. **Visual Studio Code**. Repositório no GitHub. Disponível em: <https://github.com/microsoft/vscode>. Acesso em: 29 set. 2026.

MICROSOFT. **vscode-test-electron**. Repositório no GitHub. Disponível em: <https://github.com/microsoft/vscode-test-electron>. Acesso em: 29 set. 2026.

MICROSOFT. **Workspace Trust**. In: Visual Studio Code Documentation. Disponível em: <https://code.visualstudio.com/docs/editing/workspace-trust>. Acesso em: 29 set. 2026.

OBS PROJECT. **Backend Design**. In: OBS Studio Documentation. Disponível em: <https://docs.obsproject.com/backend-design>. Acesso em: 29 set. 2026.

OBS PROJECT. **OBS Studio**. Repositório no GitHub. Disponível em: <https://github.com/obsproject/obs-studio>. Acesso em: 29 set. 2026.

OBS PROJECT. **Plugins**. In: OBS Studio Documentation. Disponível em: <https://docs.obsproject.com/plugins>. Acesso em: 29 set. 2026.

PASERO, B.. **Migrating VS Code to Process Sandboxing**. Visual Studio Code Blog, 28 nov. 2022. Disponível em: <https://code.visualstudio.com/blogs/2022/11/28/vscode-sandbox>. Acesso em: 29 set. 2026.

QT GROUP. **Qt Framework**. Site oficial. Disponível em: <https://www.qt.io/>. Acesso em: 29 set. 2026.

RICHARDS, M.; FORD, N. **Fundamentals of Software Architecture: An Engineering Approach**. 1. ed. Sebastopol: O'Reilly Media, 2020.

RIEKEN, J.. **Resolving extension issues with bisect**. Visual Studio Code Blog, 16 fev. 2021. Disponível em: <https://code.visualstudio.com/blogs/2021/02/16/extension-bisect>. Acesso em: 29 set. 2026.